



USENIX

THE ADVANCED COMPUTING
SYSTEMS ASSOCIATION

PolarStore: High-Performance Data Compression for Large-Scale Cloud-Native Databases

Qingda Hu, Xinjun (Jimmy) Yang, Feifei Li, Junru Li, Ya Lin, Yuqi Zhou, Yicong Zhu,
Junwei Zhang, Rongbiao Xie, Ling Zhou, Bin Wu, and Wenchao Zhou,
Alibaba Cloud Computing

<https://www.usenix.org/conference/fast26/presentation/hu>

This paper is included in the Proceedings of the
24th USENIX Conference on File and Storage Technologies.

February 24–26, 2026 • Santa Clara, CA, USA

ISBN 978-1-939133-53-3

Open access to the Proceedings of the
24th USENIX Conference on File and Storage Technologies
is sponsored by



PolarStore: High-Performance Data Compression for Large-Scale Cloud-Native Databases

Qingda Hu Xinjun Yang Feifei Li Junru Li* Ya Lin Yuqi Zhou Yicong Zhu Junwei Zhang
Rongbiao Xie Ling Zhou Bin Wu Wenchao Zhou

Alibaba Cloud Computing

Abstract

In recent years, resource elasticity and cost optimization have become essential for RDBMSs. While cloud-native RDBMSs provide elastic computing resources via disaggregated computing and storage, storage costs remain a critical user concern. Consequently, data compression emerges as an effective strategy to reduce storage costs. However, existing compression approaches in RDBMSs present a stark trade-off: software-based approaches incur significant performance overheads, while hardware-based alternatives lack the flexibility required for diverse database workloads.

In this paper, we present PolarStore, a compressed shared storage system for cloud-native RDBMSs. PolarStore employs a dual-layer compression mechanism that combines in-storage compression in PolarCSD hardware with lightweight compression in software. This design leverages the strengths of both approaches. PolarStore also incorporates database-oriented optimizations to maintain high performance on critical I/O paths. Drawing from large-scale deployment experiences, we also introduce hardware improvements for PolarCSD to ensure host-level stability and propose a compression-aware scheduling scheme to improve cluster-level space efficiency. PolarStore is currently deployed on thousands of storage servers within PolarDB, managing over 100 PB of data. It achieves a compression ratio of 3.55 and reduces storage costs by approximately 60%. Remarkably, these savings are achieved while maintaining performance comparable to uncompressed clusters.

1 Introduction

Relational Database Management Systems (RDBMSs) are fundamental components of modern information technology infrastructure. In recent years, a growing number of applications have adopted cloud computing to address demands of resource elasticity and on-demand usage. Consequently, cloud service providers have developed comprehensive RDBMS solutions, such as AWS Aurora [2], Azure Hyperscale [3],

and our production system, Alibaba PolarDB [4–7]. With the rapid expansion of data stored in the cloud, storage costs have become a significant concern for users.

To address storage costs, data compression techniques have emerged as an intuitive solution [8–17]. Data compression approaches can be categorized into two main categories: software-based and hardware-based compression. Software-based compression utilizes CPU resources to execute compression algorithms and provide complex space management, while hardware-based compression offloads the compression tasks to specialized hardware. However, implementing data compression to achieve high space utilization in large-scale RDBMSs presents several challenges.

Challenge#1: performance overhead of software-based compression. Modern RDBMS business scenarios, such as online e-commerce and real-time financial transactions, demand low I/O latency. However, software-based compression faces not only computational overhead from compression operations but also a fundamental challenge in managing the mapping between original and compressed data. Since the compressed data size varies with content, systems must maintain an index to locate compressed data and handle size changes during updates. This indexing mechanism presents a critical trade-off: while fine-grained indexing could significantly improve space utilization, it introduces complex management overhead that impacts performance. This trade-off manifests differently across various RDBMS architectures: B+-Tree-based systems suffer from inherent space fragmentation due to 4KB block alignment [18, 19], while systems based on Log-Structured Merge-Trees (LSM-Tree) achieve a more compact data layout but incur substantial overhead from garbage collection [20–24]. Therefore, achieving both high space efficiency and low I/O latency remains a significant challenge.

Challenge#2: limited flexibility of hardware-based compression. Given the performance overhead of software-based compression, researchers and industry practitioners have turned to hardware-based solutions [25–30]. However, these approaches introduce new challenges in terms of flexibility.

*Junru Li is the corresponding author (rusuo.ljr@alibaba-inc.com).

In-storage compression uses computational storage drives (CSDs) [25, 26], which integrate computational capabilities to offload compression tasks to storage devices. However, CSDs are constrained by fixed 4KB input sizes due to NVMe compatibility requirements and compression algorithms that cannot be modified after production. Similarly, PCIe-attached FPGA [29] or CPU-based accelerators [30] are also limited to fixed algorithms. These limitations restrict the ability of RDBMSs to adapt key compression parameters (i.e., compression algorithms and input sizes) for diverse workload patterns. While some data requires real-time processing with minimal latency, other infrequently accessed data could benefit from more complex algorithms with larger input sizes to achieve higher compression ratios. Therefore, it is beneficial to provide flexible compression solutions that can optimize storage efficiency for cold data while meeting the low-latency requirements of latency-sensitive workloads.

To address these challenges, we propose PolarStore, a storage system for RDBMSs that co-designs hardware and software to achieve both high space utilization and low I/O latency. First, PolarStore implements a *dual-layer compression mechanism* that processes data in two stages: the software layer compresses data into 4 KB-aligned blocks, maintaining simple index management at the software level and providing flexible compression parameters (i.e., input sizes and algorithms), while the hardware layer, PolarCSD, further compresses these blocks, leveraging the existing flash translation layer (FTL) to achieve byte-granularity indexing without additional software overhead. Second, we introduce *several DB-oriented optimizations* to overcome the compression overhead. These optimizations target two critical I/O paths that directly impact user-perceived latency: redo log writes during transaction commits and page reads upon in-memory buffer pool misses.

Challenge#3: stability and scalability of compression in large-scale deployment. Deploying data compression at scale in RDBMSs presents new operational challenges. At the host level, each server is equipped with 10~12 PolarCSD devices, and resource contention (e.g., CPU and memory) or faults from software drivers can lead to host-level failures and performance fluctuations. At the cluster level, varying compression ratios across different users' data make it challenging to balance logical and physical space among storage nodes. To address these deployment challenges, we redesign the hardware based on our deployment experiences and implement a compression-aware scheduling mechanism to balance compression ratios across storage nodes, thereby ensuring both host-level stability and cluster-level space efficiency.

PolarStore is deployed across thousands of storage servers and numerous clusters within PolarDB¹. The total storage capacity has surpassed 100 PB, achieving approximately 60% storage cost reduction with a compression ratio of 3.55. Comparative experiments with the clusters without data compression

demonstrate that PolarStore maintains high performance with negligible degradation.

In summary, this paper makes the following contributions:

- PolarStore, a shared storage system that leverages a hardware-software co-design for efficient compression.
- A set of DB-oriented techniques that achieve high space efficiency without sacrificing the low I/O latency of critical operations.
- Valuable insights gained from the large-scale deployment in both CSD hardware design and cluster management, along with some practical solutions to address the encountered challenges.

This paper is organized as follows. §2 introduces cloud RDBMSs and analyzes current compression approaches in RDBMSs and their challenges. §3.1 presents our dual-layer compression design and database-oriented optimizations. §4 discusses challenges encountered in large-scale deployments and our corresponding solutions. Finally, §5 comprehensively evaluates PolarStore's space utilization and performance, including an ablation study of individual techniques.

2 Background and Motivation

This section first introduces the architecture of PolarDB, followed by a review of compression approaches in RDBMSs to reveal the motivation for developing PolarStore.

2.1 The Architecture of PolarDB

The storage-compute separation architecture has been widely adopted by leading cloud-native RDBMSs, including AWS Aurora [2], Azure Hyperscale [3], and Alibaba PolarDB [4–6] as illustrated in Figure 1. In this architecture, each database instance comprises a single read-write node (RW) responsible for handling both read and write requests, multiple read-only nodes (RO) dedicated to processing read-only queries, and a shared storage system. For write operations, the RW node retrieves the necessary pages from the shared storage, generates redo logs, and subsequently transmits these logs to the corresponding storage nodes. The storage nodes, which provide redundancy and high availability, ensure the durability of these redo logs and asynchronously apply them to generate updated pages. To maintain transactional consistency, the RW node synchronizes transaction-related information in redo logs (i.e., the log record of transaction begins and transaction commits) to the RO nodes. Each RO node parses these redo logs to generate readviews for read transactions and maintains a local LSN (LSN_i), representing the progress of log parsing. During read operations, RO nodes fetch required pages from storage nodes based on their LSN_i. In the background, storage nodes track the minimum LSN across all RO nodes LSN_{min} and apply redo logs up to this point. This design delegates page generation to storage nodes instead of having RW nodes generate and transmit pages, significantly

¹ See official document [1] for details on enabling compression feature.

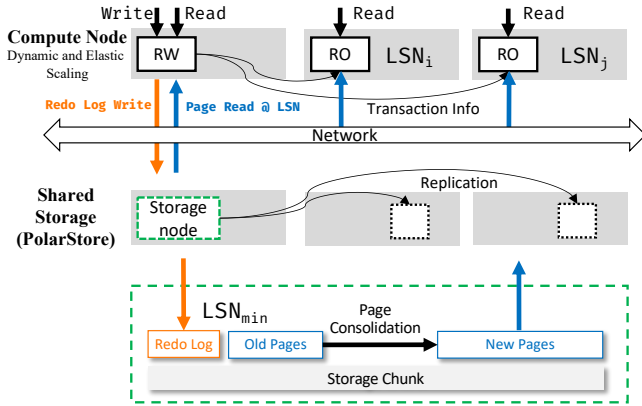


Figure 1: Architecture of PolarDB.

reducing network bandwidth consumption while enabling instant startup of computing nodes during recovery or scaling operations.

Based on this architecture, cloud computing vendors provide flexible mechanisms of adjusting instance specifications (including CPU cores and memory of RW/RO nodes) based on dynamic use-case requirements, without any data migration overhead. This capability significantly reduces the cost of computing resources for users. However, storage costs remain a more important consideration for users, especially during periods of low activity. Therefore, we introduce data compression in our RDBMSs to reduce storage costs.

2.2 Data Compression in RDBMSs

Designing compression systems for RDBMSs requires careful consideration of the following two fundamental aspects.

Index granularity. The size of compressed data varies according to the content, even when the original data size is fixed. Therefore, systems must maintain an index to locate compressed data, and when the size changes (e.g., due to updates), allocate new storage space and update the index accordingly. This indexing mechanism presents a critical *trade-off between space utilization and performance*. Large index granularity (e.g., 4 KB-level) simplifies management but wastes storage space. For example, if a 16 KB page compresses to 1 KB, but the index granularity is 4 KB, 3 KB of space is wasted. Our experiments with a 408.37 GB dataset compressed using zstd show that 4KB index granularity consumes approximately 80.5% more space than byte-level index granularity (Figure 2a). However, while finer-grained indexing improves space efficiency, it introduces complex space management that degrades performance.

Flexibility in compression parameters. The compression ratio is influenced by both the input block size and the chosen compression algorithm. Larger blocks provide more opportunities to identify repeating patterns, thereby achieving better compression ratios. Our experiments demonstrate that 1 MB blocks achieve a compression ratio of 6.85, while 4 KB blocks

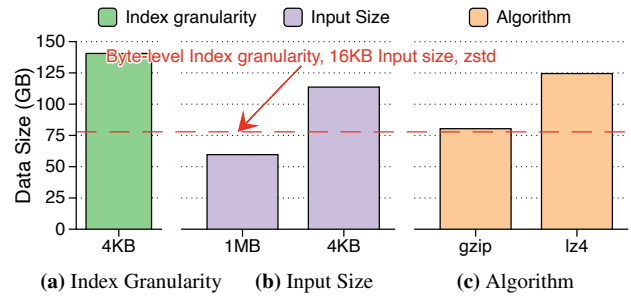


Figure 2: Compressed storage sizes of a 408.37GB dataset under different configurations. Including index granularity, input size, and algorithm. Red line: byte-level indexing, 16KB input size, and zstd, achieving 5.24× compression ratio.

achieves only 3.59 (Figure 2b). Similarly, for compression algorithms, more advanced algorithms like Zstandard (zstd) consistently outperform simpler ones like lz4 (Figure 2c). However, larger blocks and complex algorithms introduce performance overhead through I/O amplification and increased processing latency, respectively. An effective system therefore needs to dynamically adjust these parameters based on access patterns: employing larger input sizes and complex algorithms for infrequently accessed cold data, while utilizing smaller compression input sizes with simpler algorithms for hot data and data on critical paths. However, this presents another *trade-off between space utilization and performance*: while optimizing compression ratios, the flexibility in compression parameters introduces significant challenges for hardware acceleration implementation.

Current compression approaches in RDBMSs, whether software-based or hardware-based, fail to effectively balance these trade-offs between space utilization and performance.

2.2.1 Software-based Compression

We analyze three software-based compression approaches implemented at different layers: B+-Tree and LSM-Tree at the database level, and log-structured storage at the storage level. Although these approaches offer flexibility in compression parameters, they all struggle to balance space efficiency with the resulting index management overhead.

Compression in B+-Tree. Compression in B+-Trees (A in Figure 3) is implemented through two distinct strategies. The first strategy integrates compression directly into the tree structure by mapping each 16 KB page to multiple 4 KB blocks based on the compressed page size. This approach handles updates by appending uncompressed data at the page's end and performs compression during page merge or split operations (e.g., InnoDB's table compression feature [18]). The second strategy preserves the original tree structure while compressing pages prior to disk writes, utilizing file system hole-punching for space reclamation (e.g., InnoDB's page compression feature [19]). However, B+-Trees suffer from inherent fragmentation, typically reserving approximately 20%

Compression Approach	Input Size $\Rightarrow$	Index Granularity	Algorithm
Compression in B+-Tree [18, 19]	Flexible (16KB DB Page) $\Rightarrow$	4KB File Blocks	Flexible
Compression in LSM-Tree [22–24]	Flexible (16KB DB Page) $\Rightarrow$	Bytes, GC Overhead	Flexible
Compression in Log-structured Block Storage [31, 32]	Flexible (16KB Segment), I/O Amplification $\Rightarrow$	Bytes, GC Overhead	Flexible
In-Storage Compression [26, 28, 33]	4KB LBA $\Rightarrow$	Bytes	Inflexible
Dedicated Compression Accelerators [27, 29, 30]	-	-	Flexible
PolarStore	Flexible (16KB DB Page) $\Rightarrow$ 4KB LBA $\Rightarrow$	Bytes	Flexible

Table 1: Comparison of data compression approaches in cloud RDBMSs. Weaknesses highlighted in red.

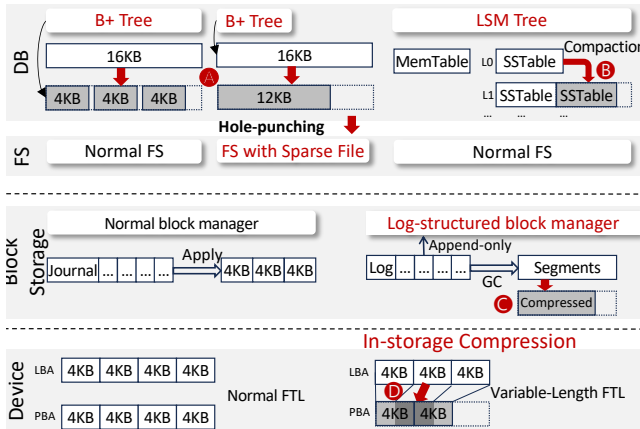


Figure 3: Data compression approaches in cloud RDBMSs. The comparison of these approaches is shown in Table 1.

to 50% of page space to accommodate future insertions [28]. Although this unused space can be compressed, the 4 KB block granularity for indexing the compressed data still causes fragmentation and leads to suboptimal space efficiency.

Compression in LSM-Trees. LSM-Trees (B in Figure 3), exemplified by RocksDB [22], LevelDB [23] and OceanBase [24], integrate compression into their compaction mechanism. During compaction operations, the data is compressed before being written to new SSTables, then the system updates its index to track the compressed data blocks. While this approach achieves better space efficiency than B+-Trees by reducing fragmentation, it introduces substantial garbage collection overhead. This overhead not only costs CPU resources but also competes with normal operations for I/O resources [20, 34–36].

Compression in log-structured block storage. Log-structured block storage systems (C in Figure 3), such as Alibaba’s Pangu [31, 32], employ compression during segment compaction. While these systems encounter challenges similar to LSM-Trees, they suffer an additional performance penalty due to misalignment between compression units and database access units. When a database page spans multiple compressed units, accessing a single page requires multiple read and decompression operations.

2.2.2 Hardware-based Compression

Hardware-based approaches address performance overhead by offloading compression tasks to specialized accelerators, although this typically comes at the cost of reduced flexibility in compression input size and algorithm selection.

In-storage compression. Computational storage devices (CSDs, D in Figure 3) integrate computational and memory resources within storage devices [26, 28, 33, 37–40]. They can offload both compression/decompression tasks and index management to in-storage components, with the latter handled by the flash translation layer (FTL). By extending the FTL to support non-aligned Physical Block Addresses (PBA) and leveraging existing garbage collection mechanisms, CSDs enable efficient, fine-grained indexing without software overhead. However, their in-storage compression is constrained by fixed 4KB compression input sizes (for NVMe compatibility) and immutable compression algorithms set during manufacturing, limiting their flexibility for dynamic data characteristics and access patterns.

Dedicated compression accelerators. Various hardware accelerators exist for compression/decompression, including PCIe-attached FPGAs/ASICs [27, 29] and CPU-based accelerators such as Intel QAT [30]. While they effectively reduce computational overhead, they do not address the fundamental challenge of index management.

In summary, as illustrated in Table 1, existing RDBMS compression approaches face different challenges: software-based approaches offer flexibility in compression parameters but incur performance overhead for byte-level indexing, while hardware-based approaches improve performance but lack adaptability to varying workloads. This limitation motivates us to develop a novel approach that combines the advantages of both software-based and hardware-based approaches.

3 Design of PolarStore

3.1 Overview and Key Ideas

In this paper, we propose PolarStore, a shared storage system for RDBMSs that achieves both high space utilization and high performance through two key ideas:

Dual-layer compression. PolarStore implements a novel dual-layer compression architecture that combines software compression with in-storage compression. The software layer compresses user data into 4 KB-aligned blocks, which are sub-

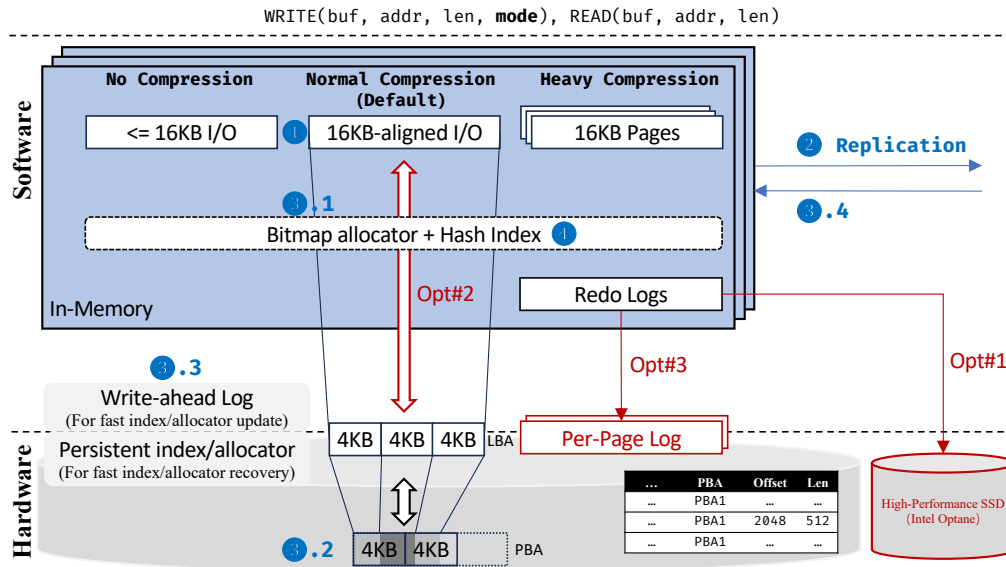


Figure 4: Overview of PolarStore. There are two innovations: dual-layer compression with hardware-software co-designing and DB-oriented I/O optimizations (Opt). This figure also shows the workflow of a 16 KB write with normal compression.

sequently compressed by PolarCSD into byte-aligned blocks. This design effectively addresses two fundamental challenges:

- For the granularity of the index, PolarStore achieves byte-level indexing granularity by leveraging the FTL’s existing garbage collection mechanisms. This design eliminates additional space management overhead in software, as the software layer only needs to manage 4 KB-aligned blocks.
- For compression parameter flexibility, the software layer enables selection of compression input sizes and algorithms for different workloads.

DB-oriented optimizations. While compression operations inherently introduce computational latency to I/O operations, only two specific I/O operations directly impact user-perceived performance in RDBMSs: redo log writes and page reads. Redo log writes are critical during transaction commits, where storage nodes must persist log records and synchronize with replicas to ensure durability and high availability. Page reads become performance bottlenecks when compute nodes need to fetch pages absent from their buffer pools. Guided by these insights, PolarStore implements targeted optimizations for these critical operations, prioritizing their performance even at the cost of reduced efficiency in other noncritical operations. These optimizations include: utilizing high-performance devices to bypass compression for redo log writes (Opt#1), implementing an adaptive algorithm selection for accelerating page reads (Opt#2), and introducing a per-page log mechanism to reduce their read amplification (Opt#3).

The remainder of this section details its core components: the dual-layer compression mechanism in §3.2 and the three DB-oriented optimizations in §3.3.

3.2 Dual-layer Compression

We present PolarStore’s architecture through three key components: software design (§3.2.1), hardware implementation (§3.2.2), and the flexible compression interface (§3.2.3).

3.2.1 Lightweight Compression in Software

As illustrated in Figure 4, PolarStore’s dual-layer compression architecture exposes a block interface to the upper layer, with write operations controlled by a compression mode flag. To demonstrate the design, consider a 16 KB write request workflow. Upon receiving such a request, the storage node (acting as the leader) first compresses the data into multiple 4 KB blocks (❶). For fault tolerance, PolarStore implements 3-way Raft replication, where the leader forwards the compressed data to two replica nodes (❷). The write request achieves commitment only after the data has been persisted on a majority of replicas. To ensure durability, both leader and follower nodes execute three steps: allocate space for compressed blocks (❸.1), write them to their CSDs (❸.2), and record index updates in their write-ahead logs (❸.3). After receiving sufficient acknowledgments from followers (❸.4), the leader updates its in-memory index cache to make the changes visible and signals completion to the upper layer (❹).

PolarStore’s space management architecture comprises two primary components: a space allocator and a hash table index. The space allocation mechanism operates at two levels: a centralized allocator manages space at 128 KB granularity for each storage device, while individual logical chunks employ bitmap allocators for fine-grained 4 KB space management. PolarStore implements a hash table index to maintain mappings between uncompressed 16 KB addresses and their

corresponding compressed 4 KB addresses. While the global allocator persists its state through in-place updates, both the bitmap allocator and hash table index operate in memory. Their modifications are logged in the write-ahead log, which serves exclusively for recovery purposes.

3.2.2 In-storage Compression: PolarCSD

This part presents the design overview of PolarCSD, focusing on key components essential to understand the dual-layer compression mechanism. In §4.1, we discuss insights derived from our first-generation implementation that guided the evolution of our new-generation architecture.

PolarCSD exposes standard NVMe interfaces to the software layer and implements the gzip algorithm with compression level 5, which has been demonstrated to achieve optimal performance in hardware acceleration scenarios [29]. The space management system of PolarCSD extends the traditional page-mapping FTL architecture. While conventional page-mapping FTL maintains fixed-length (4 KB) mappings between Logical Block Addresses (LBA) and Physical Block Addresses (PBA), PolarCSD introduces variable-length index entries that support mappings from 4KB-aligned LBA to byte-level PBA. In our implementation, PolarCSD augments each mapping entry with 12-bit *length* and *offset* fields to specify compressed data positions within a 4 KB boundary. This enhancement adds 3 bytes to each index entry, increasing the memory footprint from the original 5 bytes (for the basic information of L2P mapping) to 8 bytes per entry.

To maintain compatibility, we set PolarCSD with a logical capacity of 7.68 TB, aligned with the standard capacity of mainstream enterprise SSDs. The physical NAND flash capacity is dimensioned based on compression ratios in our target workloads. Our comprehensive evaluation of the gzip algorithm (compression level 5), configured to process 4 KB-aligned inputs with byte-level output granularity, demonstrates an average compression ratio of 2.4 across diverse datasets. Based on this compression efficiency, PolarCSD is provisioned with at least 3.2 TB of physical NAND flash space to support the logical capacity of 7.68 TB and the 2.4 compression ratio.

3.2.3 Flexibility of Interface

Write interface. The storage layer extends its write interface with a flag field that supports three compression modes: normal compression, no compression, and heavy compression.

Normal compression: Following the workflow presented in §3.2.1, this default mode mandates that I/O operations be aligned to database page boundaries (typically 16 KB or 8 KB). Any non-aligned operation automatically reverts to the no-compression mode.

No compression: This mode serves two specific scenarios: handling non-page-aligned I/O operations and processing user-designated uncompressed pages. The software bypasses compression and writes the data directly to PolarCSD. When writing to a part of a previously compressed range, PolarStore must decompress the existing data, allocate new storage space,

and write the uncompressed data to PolarCSD.

Heavy compression: This mode is specifically designed to restore and compress a range of data (i.e., archiving operations). Unlike other modes, this interface does not write new data to the storage. Instead, it processes the entire write range as a single compression unit and uses high-level compression configuration to achieve higher compression ratios. It first reads and decompresses any existing compressed pages within the range, then merges them into a single segment and recompresses this consolidated segment for optimal compression ratios. The compressed segment is stored contiguously, with each index entry in the hash table maintaining both the address of the compressed segment and the page offset within that segment. While this approach may introduce I/O amplification during random access, such overhead is negligible for archived and snapshot pages in databases, which are typically accessed sequentially during analytical processing and backup/recovery operations, and are infrequently accessed. A temporary buffer for decompressed segments effectively optimizes such sequential access patterns.

Read interface. Unlike the write interface, the read interface does not require additional parameters. The system maintains three essential attributes in each index entry: compression status, compression algorithm used, and segment information for heavily compressed pages. These attributes guide the system in determining the appropriate read range and decompression strategy during read operations.

3.3 DB-oriented Optimizations

Building upon the architectural overview, we introduce three DB-oriented optimizations that minimize compression overhead by exploiting database-specific access patterns, with a focus on reducing the latency of redo writes and page reads. These optimizations include (Figure 4):

- (Opt#1) Utilizing high-performance devices to bypass compression for latency-sensitive redo log writes (§3.3.1)
- (Opt#2) Implementing adaptive algorithm selection between lz4 and zstd to balance compression ratios and read latency (§3.3.2)
- (Opt#3) Introducing a per-page log mechanism that leverages physical-logical space decoupling to reduce read amplification during page accesses (§3.3.3)

3.3.1 Avoiding Compression for Log Write

Redo log persistence is critical for transaction commit latency. To minimize this latency, PolarStore bypasses all compression for redo logs: it employs the no-compression flag to skip software compression and leverages high-performance storage to avoid hardware compression. In our implementation, each server incorporates an Intel Optane SSD, which delivers superior and stable performance with limited space. Initially, these devices were dedicated to storing WAL for in-memory data structures (the allocator and index), and their usage has now been extended to include redo logs as well. The use of Optane SSDs for redo logs is a good match for their characteristics:

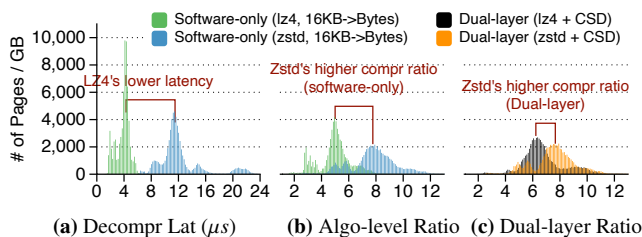


Figure 5: Performance and compression ratio analysis of LZ4 and Zstd. (a) Decompression latency confirms zstd’s higher computational cost. (b) At the software level, zstd offers a superior compression ratio. (c) However, in our dual-layer system, zstd’s advantage is substantially diminished.

the limited capacity of these devices is sufficient for redo logs, which are typically small and can be reclaimed after pages are flushed.

3.3.2 Low-latency I/O and Decompression for Page Read

The I/O latency of page reads directly impacts query latency when requested pages are absent from the in-memory buffer pool. These page reads include two parts: storage read latency and decompression latency. To reduce these latencies, we introduce an adaptive algorithm selection mechanism. Our adaptive algorithm selection mechanism is motivated by two key observations.

First, conventional wisdom holds that zstd, despite its longer decompression times (Figure 5a), achieves superior compression ratios compared to lz4 (Figure 5b). In the algorithm layer, there is a simple trade-off between performance and compression ratios. However, as shown in Figure 5c, our comprehensive evaluation reveals an interesting phenomenon in the dual-layer compression setting: its compression advantage significantly diminishes from 58.9% at the algorithm level to merely 9.0% after hardware compression. This substantial reduction occurs because hardware gzip, which also employs Huffman encoding, can effectively further compress lz4’s output (which lacks Huffman encoding) while gaining minimal additional compression on zstd’s output (which already incorporates Huffman encoding). This observation reveals that: *for some pages, using lz4 offers both lower decompression latency and a competitive compression ratio compared to using zstd.*

Second, the 4 KB I/O alignment of compressed pages creates another powerful optimization opportunity. Even a marginal size reduction at the software layer from zstd can be enough to save an entire 4 KB I/O block. For instance, if lz4 compresses a 16 KB page to 4097 bytes (requiring 8 KB of I/O in data write/read) while zstd achieves 4096 bytes (requiring only 4 KB of I/O), the I/O savings in data read can easily outweigh zstd’s higher decompression overhead. This observation demonstrates a counter-intuitive scenario where: *for certain pages, using zstd can achieve a lower total page read latency than lz4.*

Taken together, these observations challenge the notion of a simple trade-off between latency and compression ratio in using lz4 or zstd. Instead, they reveal the potential for a “win-win” outcome: the compression algorithm is no longer a static choice between two competing algorithms, but a dynamic decision to pick the clear winner for each specific page. Therefore, PolarStore implements a page-level algorithm selection mechanism during page writes, as shown in Algorithm 1. It evaluates both algorithms’ 4KB ceiling-aligned compressed sizes and decompression latencies (Line 6–8). If the ratio of saved data size (i.e., the benefits of zstd) to increased decompression latency (i.e., the overhead of zstd) exceeds a threshold (Line 15), PolarStore switches to zstd for this page; otherwise, it stays with lz4 to maintain low decompression overhead.

Algorithm 1: Selection of lz4 and zstd

```

1 Function page_compression(page):
2   if CPU_utilization > 20% then
3     ptr ← lz4()
4     return "lz4", ptr
5   else if update_percent > 30% then
6     // lz4_sz, zstd_sz: 4KB ceiling-aligned
7     lz4_ptr, lz4_sz, lz4_lat ← lz4()
8     zstd_ptr, zstd_sz, zstd_lat ← zstd()
9
10    // extra decompression latency of zstd: μs
11    overhead ← zstd_lat - lz4_lat
12    // saved storage space of zstd: bytes
13    benefit ← lz4_sz - zstd_sz
14
15    if benefit / overhead > 300B/μs then
16      return "zstd", zstd_ptr
17    else
18      return "lz4", lz4_ptr
19  else
20    ptr ← last_used_algorithm()
21    return last_used_algorithm, ptr

```

This threshold is set to 300 B/μs based on storage I/O characteristics: saving 4 KB of I/O typically reduces read latency by 12~14 μs, translating to approximately 300 B/μs. When zstd’s storage savings per additional microsecond of decompression time exceed this threshold, the I/O latency reduction outweighs the increased decompression overhead, making zstd the better choice.

It is important to note that this selection occurs during page writes, which are out of the critical path of user queries. However, the process still consumes CPU resources, which must be carefully managed to avoid impacting overall system throughput. To minimize the selection overhead, this mecha-

nism is triggered only in two cases: during initial page writes, or when the database layer estimates page updates exceed 30% based on the log size. Additionally, the selection process only runs when CPU utilization is low, ensuring that this selection does not become a performance bottleneck.

3.3.3 Mitigating the Tail Latency for Page Read

We leverage CSD's space decoupling feature to address the tail latency issue in page read operations.

For page reads, both the database buffer pool and storage software memory cache help avoid storage accesses and decompression overhead. However, when pages are not cached in memory, I/O becomes unavoidable. Three scenarios may occur: (i) the page exists but is not cached, (ii) the page does not exist but its redo logs are cached, and (iii) the page does not exist and some of its redo logs are not cached. The third case commonly occurs when a RO node's LSN falls behind due to high load or network issues, preventing the storage node from recycling redo logs and leading to cache overflow. While the first two cases require only a single page read, the third case suffers from read amplification, requiring multiple random reads to retrieve redo logs. As illustrated in Figure 6a, to generate page@6, the storage node must read both log_1 and log_3 from storage, which may reside in different 4KB address ranges. These scattered reads significantly contribute to slow tail latencies.

Design with space decoupling. CSD decouples logical space allocation from physical space utilization, allowing software to manage logical space at 4 KB granularity while utilizing physical space more flexibly. This enables implementing sparse data structures without I/O amplification or space waste [26, 28]. PolarStore introduces a per-page log mechanism. The key is co-locating all redo logs of each page within a dedicated 4 KB log space when the log is evicted from the in-memory cache. As shown in Figure 6b, when the redo log is evicted from memory (this occurs before receiving a read request for page@6), the storage node pre-merges both log_1 and log_3 into the per-page log space. This allows retrieving all necessary logs in a single read, when the storage node receives the read request for page@6, significantly reducing read amplification and improving tail latency. This solution, allocating an additional 4 KB log space for each 16 KB page, is only feasible with CSD's space decoupling feature. Implementing such a design on conventional SSDs would incur approximately 25% space amplification.

4 Large-Scale Deployment of PolarStore

We initially deployed our system across 11 clusters with 500 hosts, containing a total of 6000 PolarCSD devices. Through this large-scale deployment, we identified two critical challenges: host-level stability and cluster-level resource utilization. In §4.1, we address the host-level stability challenge by summarizing lessons learned from the first generation hardware (PolarCSD1.0) and redesigning a new generation device

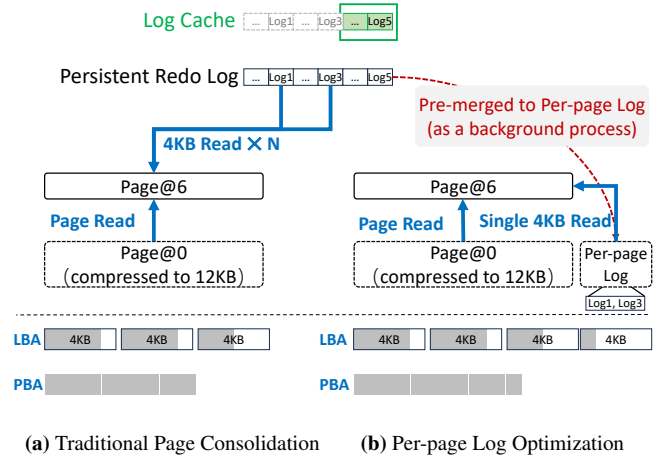


Figure 6: Workflow of page consolidation. (a) Traditional method: scattered redo logs lead to read amplification and high tail-latency. (b) Per-page log optimization: proactively co-locating logs in the background enables page consolidation with a single I/O.

(PolarCSD2.0). In §4.2, we tackle the cluster-level resource utilization challenge by introducing a compression-aware scheduling technique that effectively balances chunks with varying compression ratios across the cluster.

4.1 Host-Level Stability

The first generation of PolarCSD was designed with an open-channel architecture (host-based FTL) [36, 41], which offered advantages in rapid development and flexible space management. While we initially dedicated specific CPU cores to FTL tasks to minimize its impact on software performance, our long-term deployment of 12 devices per host revealed significant challenges in system stability and operational costs.

4.1.1 Lessons in Host-level Stability

During an eighteen-month period, there were 26 occurrences of slow I/O (i.e., exceeding 1 second) caused by PolarCSD1.0. Among these, 6 slow I/O events exceeded 10 seconds in latency and lasted for more than 10 minutes, adversely affecting the user's business and significantly increasing the maintenance overhead. These problems arise from two factors: resource contention and expanded fault domain for host-based FTL driver.

Resource contention for host-based FTL. The host-based FTL consumes host memory and CPU resources. This overhead becomes more severe when supporting variable-length address mapping and multiple drives. We observed 12 occurrences of slow I/O caused by memory contention and 9 occurrences caused by CPU contention. Each PolarCSD1.0, with a logical capacity of 7.68TB, requires 15.36 GB ($7.68TB \times 8B/4KB$) of memory for FTL. With 12 devices per storage node, the total memory consumption reaches approximately 184.32 GB. This significant memory footprint causes resource

pressure, triggering aggressive memory reclamation by the system that degrades application performance. Further, the host-based FTL for each PolarCSD1.0 requires approximately 2 dedicated physical CPU cores to maintain performance under high-pressure workloads. Consequently, each server needs 24 physical CPU cores, and these FTL threads may interfere with the original storage software, leading to I/O jitters.

Expanded fault domain. As a new product, the open-channel driver of PolarCSD contained some undiscovered bugs. When triggered by a single device, they could affect the entire server. Our observation showed that all 5 long-lasting slow I/O occurrences were caused by such kernel driver bugs.

As a temporary solution to these problems, we had to disable software compression in our dual-layer compression design and limit the deployment to 10 devices per server, ensuring sufficient resources for the host-based FTL. However, this compromise reduced our storage density due to fewer devices per server (from 12 to 10) and the loss of software compression benefits.

4.1.2 Solution: PolarCSD2.0

Based on the lessons from PolarCSD1.0, we design a new generation CSD (PolarCSD2.0) that addresses the above stability issues through several key improvements:

First, PolarCSD2.0 abandons the open-channel architecture and reverts to conventional device-managed FTL, where the embedded ARM cores handle LBA-to-PBA (L2P) mapping and background operations. This return to traditional architecture eliminates the resource contention issues of host-based FTL and contains FTL failures within individual devices, preventing them from affecting the entire host system. Second, to improve storage density, we increase the NAND flash capacity in PolarCSD2.0 to 3.84 TB (using 4 TB NAND flash with 4% over-provisioning), enabling 20% more compressed data storage per device. Third, to maintain the overall compression ratio of 2.4 with this increased storage capacity, we optimize the FTL mapping structure to avoid additional memory consumption. To reduce FTL memory consumption, we redesigned the L2P mapping entry in PolarCSD2.0. While PolarCSD1.0 used an 8-byte entry, PolarCSD2.0 reduces this to 7 bytes. This memory saving is achieved by coarsening the physical offset granularity from a single byte to 16 bytes, which allows the required offset and length metadata to be encoded in 2 bytes instead of 3. This optimization effectively addresses the memory requirements from the increased storage capacity and allows us to expose 9.6 TB of logical space for each PolarCSD2.0 device. Finally, following the industry trend, PolarCSD2.0 adopts PCIe 4.0 to enhance I/O performance.

4.1.3 The Evaluation of PolarCSD2.0

We first conduct basic performance testing. For fair comparison, we use Intel P4510 (PCIe 3.0) and P5510 (PCIe 4.0) as the baseline for PolarCSD1.0 (PCIe 3.0) and PolarCSD2.0 (PCIe 4.0) respectively, matching their respective PCIe interfaces. Figure 7 presents the latency under workloads with dif-

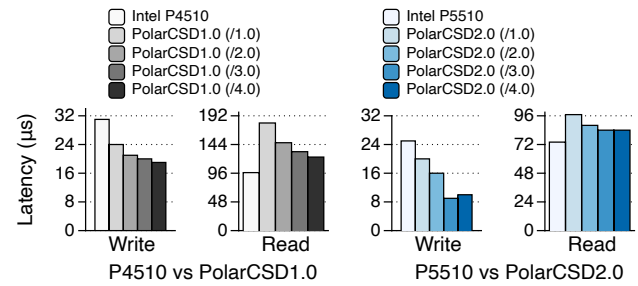


Figure 7: Average latency of PolarCSD and standard SSDs under different compression ratios. Workload: 16KB I/O, queue-depth=1. Target compression ratios: 1.0, 2.0, 3.0, 4.0 (configured via FIO [42]).

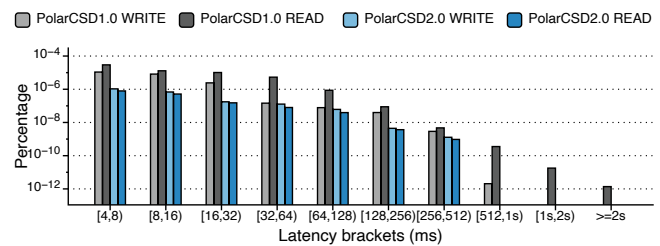


Figure 8: Distribution of device latency (≥ 4 ms) in production with 4~16KB READ and WRITE operations.

ferent compression ratios. The results show that PolarCSD1.0 achieves lower write latency but higher read latency compared to P4510. We also observe that larger compression ratios lead to lower latencies, as less data needs to be written to or read from NAND flash.

After large-scale deployment in our production environment, we observe significant improvements in system stability. First, there is no slow I/O caused by CPU/memory contention, and the failure of a single device does not impact the whole host. Further, the tail latency of PolarCSD2.0 is also significantly reduced. Figure 8 shows the latency distribution of these two generations of PolarCSD in our production environment during 7 days. We only show the percentage of I/Os with latency larger than 4ms in this figure, and we observe that only 7.91×10^{-7} read and 1.05×10^{-6} write latency of PolarCSD2.0 are larger than 4ms. In contrast, PolarCSD1.0 shows much higher percentages (2.9×10^{-5} and 4.0×10^{-5} , respectively), approximately 36.7 times and 38.8 times higher than PolarCSD2.0.

4.2 Cluster-level Space Management

During our large-scale deployment, we discovered that the original data scheduling strategy fails to effectively handle compressed data, leading to suboptimal resource utilization.

4.2.1 Lessons in Cluster-level Resource Utilization

Initially, our clusters employed a simple scheduling strategy based solely on logical space usage: new chunks were allo-

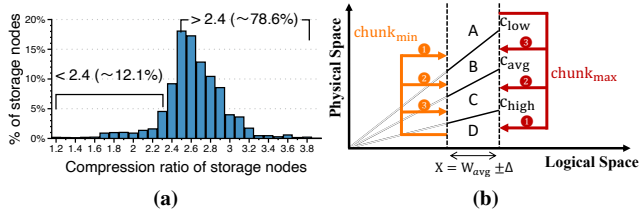


Figure 9: (a) Distribution of compression ratio in a cluster, and (b) Compression-aware scheduling algorithm.

cated to storage nodes with the lowest logical space usage. When a node's logical space usage exceeded the average usage w_{avg} by 10%, chunks would be migrated to nodes with the lowest logical space usage. Nodes exceeding 75% space usage (any of logical space usage or physical space usage) were blocked from receiving new chunks, and when all nodes reached this threshold, manual intervention was required to add new storage servers. During deployment, we identified two major limitations in our original space management:

Inaccurate physical space monitoring. The software can query devices for physical space usage, but the lack of TRIM operations leads to inaccurate measurements. When our software allocator frees space, it only updates space management metadata without actually releasing physical space via TRIM operations, and therefore devices remain unaware of these releases, causing reported physical space usage to exceed actual usage. After enabling TRIM operations upon space deallocation, the monitored physical space decreased by 3% on average.

Compression ratio imbalance. The original strategy failed to account for varying compression ratios across different chunks, leading to significant space waste. When the cluster became full, we observed an interesting phenomenon: some storage nodes reached their 75% logical space limit while their physical spaces remained underutilized, while other nodes showed the opposite pattern. Simply redistributing chunks between these two types of nodes could have increased the cluster's effective capacity without adding new servers. Our analysis of a full cluster running PolarCSD1.0 revealed the extent of this imbalance (Figure 9a): 12.1% of storage nodes had below-average compression ratios, wasting 1.72% of total logical space, while 78.6% had above-average ratios, wasting 9.17% of total physical space.

4.2.2 Solution: Compression-aware Scheduling

To balance both logical and physical space usage, we developed a compression-aware scheduling strategy. As shown in Figure 9b, storage nodes can be visualized on a two-dimensional plane with logical space (x-axis) and physical space (y-axis). Since the cluster allocates new chunks to nodes with the lowest logical space usage, most nodes are distributed between $x = w_{avg} \pm \Delta$, where w_{avg} is the average logical space usage. Our strategy aims to maintain compression ratios within a range $[c_l, c_h]$, where $c_l < c_{avg} < c_h$ and

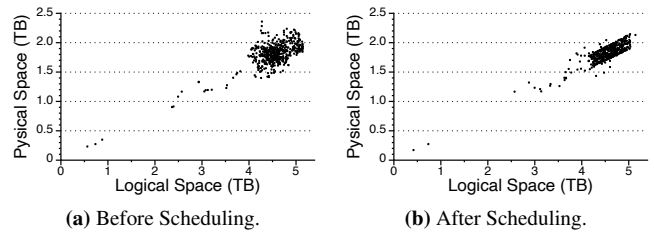


Figure 10: Logical-to-physical space mapping of storage instances before and after scheduling. Clusters with hardware-only compression (PolarCSD1.0).

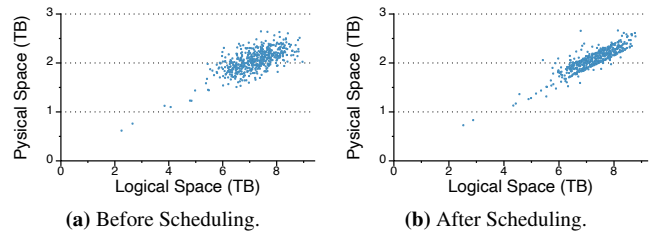


Figure 11: Logical-to-physical space mapping of storage instances before and after scheduling. Clusters with dual-layer compression (PolarCSD2.0 and software compression).

c_{avg} is the average compression ratio. These values divide the operational region into four zones: Zone A (high physical, low logical usage), Zone B (balanced usage, below average), Zone C (balanced usage, above average), and Zone D (low physical, high logical usage). For nodes in Zone A, PolarStore migrates chunks with minimum compression ratios to nodes in zones D, C, or B (in order of preference). Conversely, for nodes in Zone D, chunks with maximum compression ratios are migrated to nodes in zones A, B, or C.

4.2.3 Scheduling Results in Production Clusters

The selection of c_l and c_h is a trade-off between the scheduling results and the number of scheduling tasks. Generally, lower c_l and higher c_h result in fewer tasks. We determine these parameters through offline simulations for each cluster, targeting the parameters completion within one day. We evaluated our strategy on two production clusters: C1 (using PolarCSD 1.0 without software compression) and C2 (using PolarCSD 2.0 with software compression). Figure 10a and Figure 10b show the distribution of logical and physical spaces in C1 before and after scheduling, while Figure 11a and Figure 11b show results for C2. After scheduling, storage nodes converged into a distinct quadrilateral region, with over 90% of nodes in C1 achieving compression ratios between 2.2 and 2.7, and 87.7% of nodes in C2 between 3.15 and 3.85, demonstrating effective compression ratio balance.

5 Evaluation

In this section, we seek to answer the following questions:

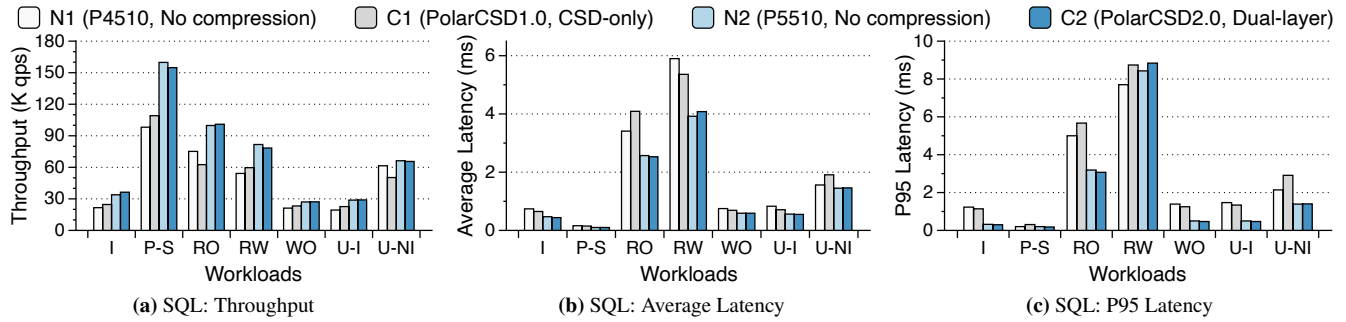


Figure 12: Overall performance. Configurations: Sysbench [43], 16 threads in a single client. Workloads: **I**: Insert, **P-S**: Point Select, **RO**: OLTP-Read-Only, **RW**: OLTP-Read-Write, **WO**: OLTP-Write-Only, **U-I**: Update-Index, **U-NI**: Update-Non-Index.

- What are the space savings and performance benefits of PolarStore in large-scale production deployments (§5.1)?
- How much does each technique contribute to PolarStore’s overall effectiveness (§5.2)?
- How does PolarStore compare with existing software compression approaches (§5.3)?

5.1 Cluster-level Evaluation

Our production environment consists of over 500 storage servers with more than 6000 first-generation computational storage devices (PolarCSD1.0) and over 1200 storage servers with over 14400 second-generation computational storage devices (PolarCSD2.0), respectively. We maintain two types of clusters: The first type, represented by cluster C1, uses PolarCSD1.0, where we had to disable software compression and two DB-oriented optimizations (i.e., the selection mechanism of lz4/zstd and the per-page log mechanism) to mitigate resource contention issues. The second type, represented by cluster C2, uses PolarCSD2.0 with all techniques enabled. To evaluate the effectiveness of these two compressed clusters, we compare their space utilization and database performance with two normal clusters (N1 and N2) selected from our production environment. These normal clusters use contemporary Intel SSDs (P4510 in N1, P5510 in N2) and have identical hardware specifications (CPU, PCIe, and NICs) to C1 and C2 respectively. Detailed configurations are shown in Table 2.

5.1.1 Space Utilization and Cost Analysis

The cluster C1, equipped with PolarCSD1.0, achieves a 2.35 compression ratio in production. Although the hardware cost of PolarCSD1.0 is 1.45 higher than the baseline Intel P4510 (normalized as 1.00) due to additional embedded memory and accelerators, the effective cost per GB of logical storage reduces to 0.62 after compression. The cluster C2, equipped with PolarCSD2.0 and dual-layer compression, demonstrates even better results. Through hardware optimization, PolarCSD2.0’s relative cost reduces by 9% compared to PolarCSD1.0 (from 1.45 to 1.32). More importantly, with the addition of software compression, C2 achieves a significantly higher compression ratio of 3.55. This brings the effective cost per GB down to 0.37, representing about 60% reduc-

Cluster	N1	C1	N2	C2
Software	-	-	-	Compression
Hardware	P4510	PolarCSD1.0	P5510	PolarCSD2.0
Opt#1: bypass redo	-	✓	-	✓
Opt#2: lz4/zstd	-	✗	-	✓
Opt#3: per-page log	-	✗	-	✓
Scheduling	-	✓	-	✓
NAND Size	3.84 TB	3.20 TB	7.68 TB	3.84 TB
Compression Ratio	-	2.35	-	3.55
Cost/GB(Physical)	1	1.45	0.91	1.32
Cost/GB(Logical)	1	0.62	0.91	0.37
CPU	Xeon Platinum 2.5GHz		Xeon Platinum 2.9GHz	
NIC	CX-4 (25Gbps×2)		CX-6 (100Gbps×2)	
PCIe	3.0		4.0	
# of storage devices	12	10	12	12
Performance layer	Intel P4800X		Intel P5800X	

Table 2: Cluster configurations and space utilization.

tion in storage cost compared to using Intel P5510 (0.91).

5.1.2 Performance Evaluation

We use Sysbench [43] to evaluate database performance. The database compute instance runs with 8 cores and 32 GB memory, accessing data distributed across 8 storage nodes (48 chunks in total). The database size is configured to 480 GB with 60 GB data per storage node (120 GB with replication). Database requests are generated by a separate Elastic Compute Service (ECS) client using 16 threads. The limited memory configuration creates an I/O-bound environment, ideal for evaluating storage system compression/decompression performance. We evaluate throughput (Figure 12a), average latency (Figure 12b), and P95 latency (Figure 12c) across different workloads. While C1 (PolarCSD1.0) shows a 10% performance degradation compared to N1 (P4510), our latest C2 cluster (PolarCSD2.0) achieves performance parity with N2 (P5510), demonstrating the effectiveness of our hardware and software optimizations.

5.2 Ablation Study of Techniques

To quantify the contribution of each technique, we conduct ablation studies on both performance and space utilization. We run the baseline on N2 cluster with P5510 SSDs, while on C2 cluster we add optimization technique one at a time to eval-

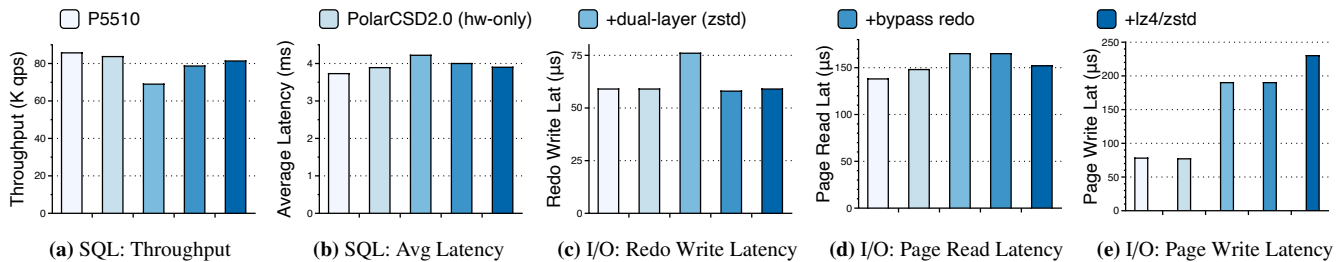


Figure 13: Impact of techniques on performance. This figure contrasts user-level performance with internal I/O latency: (a) and (b) show key user-request metrics, while (c), (d), and (e) detail the components of I/O latency at the storage layer.

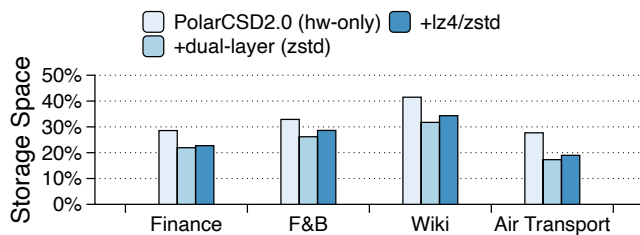


Figure 14: Impact of techniques on space utilization.

uate its impact. For performance evaluation (Figure 13), we use the identical benchmark environments as described above. All experiments use Sysbench OLTP-Read-Write workload. For space utilization (Figure 14), we use N2’s storage size as baseline and measure the relative compression ratios achieved in C2. We evaluate compression effectiveness using four production datasets that were dumped from user databases and restored in our experimental databases with permission.

Base hardware compression (PolarCSD). With only PolarCSD’s hardware compression enabled, we observe compression ratios ranging from $2.12\times$ to $3.84\times$ across different datasets, but at the cost of 7.4% throughput reduction compared to P5510 (Figure 13a). This performance degradation stems from PolarCSD’s higher read latency affecting page read operations (Figure 13d).

Dual-layer compression (+dual-layer). Adding software compression (using zstd by default) further improves compression ratios by 21.7%~50.3% but leads to a 19.6% throughput reduction compared to using hardware compression alone. This performance drop mainly occurs because compressing 16KB redo writes in the software layer slows down redo write operations, increasing their average latency (with 3-way replication and durability) from $59\mu s$ to $79\mu s$, as shown in Figure 13c.

Avoiding compression for log write (+bypass redo). By bypassing compression for redo writes, we reduce the throughput degradation to only 8.9% compared to hardware compression. Since redo logs are small and frequently recycled, this optimization maintains the compression ratio for user data.

Low-latency I/O and decompression for page read via compression selection (+lz4/zstd). When we enable the

Dataset	Finance	F&B	Wiki	Air Transport
zstd	73.1%	41.3%	52.4%	51.6%
lz4	26.9%	58.7%	47.5%	48.4%

Table 3: Distribution of selected compression algorithms (zstd vs. lz4) across different database workloads.

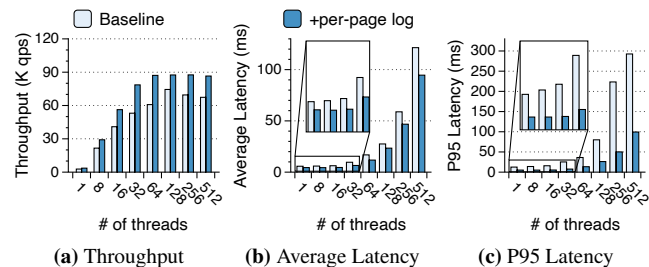


Figure 15: Performance of OLTP read-only (RO nodes) before and after using per-page log optimizations.

algorithm selection between lz4 and zstd, system throughput improves significantly, approaching the baseline performance (only 2.1% lower as shown in Figure 13a). This improvement stems from our adaptive strategy that optimally balances I/O and decompression overhead: choosing lz4 when faster decompression is beneficial, and selecting zstd when its additional compression can reduce I/O operations. This optimization reduces average page read latency by approximately $9\mu s$ compared to using zstd exclusively. While this selection mechanism increases page write latency, as shown in Figure 13e, this overhead occurs in the background and does not directly impact user operations. Note that, in our evaluation, the update is always issue the algorithm re-selection, representing the worstpage write latency. And in real workloads, algorithm re-selection is unfrequent, which does not bring high write latency. The impact on compression ratio is minimal. As shown in Figure 14, using compression selection only increases storage space by 0.7%~2.6% compared to using zstd exclusively. The distribution of pages compressed by zstd versus lz4 varies across different datasets, as shown in Table 3.

Reducing the tail latency for page read via per-page log. To evaluate the effectiveness of our per-page log design un-

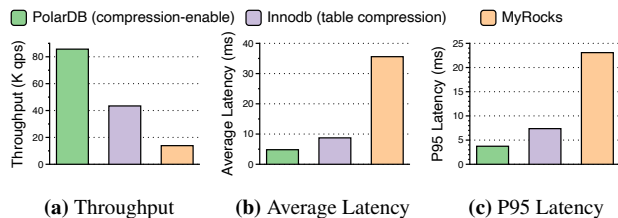


Figure 16: End-to-end performance comparison with other approaches.

der insufficient log cache conditions, we set up a two-node experiment: a read-write (RW) node and a read-only (RO) node, with the RO node intentionally lagging approximately 1s behind in LSN synchronization. This setup prevents log recycling at storage nodes and ensures log cache pressure. We direct OLTP write-only workloads to the RW node and read-only workloads to the RO node. As shown in Figure 15, we measure performance on the RO node with varying thread counts. With threads below 128, the per-page log optimization significantly reduces P95 latency by 28.9%~39.5% compared to the baseline. This improvement stems from reduced I/O amplification during page generation, as our design enables retrieving all necessary logs with a single read operation instead of multiple scattered reads. However, beyond 128 threads, the performance becomes CPU-bound at the RO node, where software queuing dominates the latency and diminishes the benefits of our I/O optimization.

5.3 Comparison with Other Approaches

We compare PolarDB that enables compression [1] with two other popular databases that implement compression at the database layer: InnoDB with table compression [18] and MyRocks with compression enabled [20]. Figure 16 shows throughput and latency comparison using Sysbench OLTP-Read-Write workload. This experiment uses the same configurations as previous ones. PolarDB demonstrates superior performance over other systems. This is because InnoDB and MyRocks consume resources in compute nodes (which are billed to users) for both space management and compression/decompression operations. These approaches force users to make a trade-off between storage and computing resources, which does not truly reduce costs. In contrast, PolarStore implements compression at the shared storage layer, which offers two key advantages: first, compression becomes completely transparent to database users without consuming their compute resources. Second, by centrally managing resources used for compression across multiple users, cloud providers can potentially achieve better resource utilization and lower operational costs.

6 Discussion

This section discusses related research directions on compression in RDBMSs and alternative space-saving approaches.

Related Directions on Data Compression. There are also

some solutions can further improve data compression in RDBMSs. First, to improve compression ratios, we can leverage table-level information to generate shared dictionaries [44, 45] for pages within the same table. This approach improves compression ratios by exploiting schema-level semantics and reducing per-page metadata overhead. Second, we can optimize page layouts to facilitate dictionary construction. Database systems store structured data with well-defined field boundaries and type information. By leveraging these properties and incorporating specialized data encoding techniques, we can achieve substantially better compression ratios compared to general-purpose algorithms. Additionally, clustering data of the same column together [14, 46, 47] can enhance compression effectiveness by exploiting column-specific patterns. Third, estimation techniques [16] can enable rapid algorithm selection to better balance performance and compression ratios. Fourth, leveraging hardware acceleration and vector instructions available in modern CPUs [30, 48] can further improve compression/decompression performance.

Alternative Space-Saving Approaches. Storage tiering [49–52] offers another approach to cost reduction. For instance, our system supports table-level archiving of cold data to object storage. Erasure coding (EC) [53–56] presents an alternative for reducing storage costs while maintaining reliability comparable to replication-based systems. However, EC is not currently suitable for our system’s redo records and remains a future research direction. Further, data deduplication [57–65] can effectively reduce storage costs, but its applicability in RDBMSs is limited since data is typically stored at the record level, making exact page-level deduplication matches rare.

7 Conclusion

This paper revisits data compression in RDBMS and presents PolarStore, a shared storage system with dual-layer compression architecture for large-scale cloud-native databases. The key contributions include a hardware/software co-designed compression mechanism that achieves both high space utilization and flexibility, database-oriented optimizations that maintain high performance on critical I/O paths, and practical solutions for large-scale deployment challenges. PolarStore has been deployed on thousands of nodes, serving tens of thousands of customers, and represents one of the largest known deployments of computational storage in production databases.

Acknowledgements

We sincerely thank our shepherd Juncheng Yang and anonymous reviewers for their valuable feedback, which significantly improved this paper. We thank the PolarDB Infrastructure Team and Alibaba Infrastructure Service (AIS) Group for their support and contributions to this work.

References

- [1] Enable the storage compression feature in PolarDB. [Online]. <https://www.alibabacloud.com/help/en/polardb/polardb-for-mysql/how-to-turn-on-storage-compression>.
- [2] Alexandre Verbitski, Anurag Gupta, Debanjan Saha, Murali Brahmadesam, Kamal Gupta, Raman Mittal, Sailesh Krishnamurthy, Sandor Maurice, Tengiz Kharatishvili, and Xiaofeng Bao. Amazon aurora: Design considerations for high throughput Cloud-Native relational databases. In *ACM International Conference on Management of Data (SIGMOD '17)*, pages 1041–1052. ACM, 2017.
- [3] Panagiotis Antonopoulos, Alex Budovski, Cristian Diaconu, Alejandro Hernandez Saenz, Jack Hu, Hanuma Kodavalla, Donald Kossmann, Sandeep Lingam, Umar Farooq Minhas, Naveen Prakash, Vijendra Purohit, Hugh Qu, Chaitanya Sreenivas Ravella, Krystyna Reisteter, Sheetal Shrotri, Dixin Tang, and Vikram Wakade. Socrates: The new SQL server in the cloud. In *International Conference on Management of Data (SIGMOD '19)*, pages 1743–1756. ACM, 2019.
- [4] Feifei Li. Cloud native database systems at alibaba: Opportunities and challenges. *Proc. VLDB Endow.*, 12(12):2263–2272, 2019.
- [5] Zongzhi Chen, Xinjun Yang, Feifei Li, Xuntao Cheng, Qingda Hu, Zheyu Miao, Rongbiao Xie, Xiaofei Wu, Kang Wang, Zhao Song, Haiqing Sun, Zechao Zhuang, Yuming Yang, Jie Xu, Liang Yin, Wenchao Zhou, and Sheng Wang. CloudJump: Optimizing cloud databases for cloud storages. *Proc. VLDB Endow.*, 15(12):3432–3444, aug 2022.
- [6] Wei Cao, Zhenjun Liu, Peng Wang, Sen Chen, Caifeng Zhu, Song Zheng, Yuhui Wang, and Guoqing Ma. PolarFS: An ultra-low latency and failure resilient distributed file system for shared storage cloud database. *Proc. VLDB Endow.*, 11(12):1849–1862, aug 2018.
- [7] Zongzhi Chen, Xinjun Yang, Mo Sha, Feifei Li, Kang Wang, Zheyu Miao, Jie Xu, Jianfeng Wang, and Sheng Wang. Cloudjump ii: Optimizing cloud databases for shared storage. In *ACM SIGMOD International Conference on Management of Data (SIGMOD '25)*, SIGMOD/PODS '25, page 336–349, New York, NY, USA, 2025. Association for Computing Machinery.
- [8] Gennady Pekhimenko, Chuanxiong Guo, Myeongjae Jeon, Peng Huang, and Lidong Zhou. TerseCades: Efficient data compression in stream processing. In *USENIX Annual Technical Conference (USENIX ATC '18)*, pages 307–320. USENIX Association, jul 2018.
- [9] Xiaokang Hu, Fuzong Wang, Weigang Li, Jian Li, and Haibing Guan. QZFS: QAT accelerated compression in file system for application agnostic and cost efficient data storage. In *USENIX Annual Technical Conference (USENIX ATC '19)*, pages 163–176. USENIX Association, jul 2019.
- [10] Daniel Reiter Horn, Ken Elkabany, Chris Lesniewski-Lass, and Keith Winstein. The design, implementation, and deployment of a system to transparently compress hundreds of petabytes of image files for a File-Storage service. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI '17)*, pages 1–15. USENIX Association, mar 2017.
- [11] Bo Mao, Hong Jiang, Suzhen Wu, Yaodong Yang, and Zaifa Xi. Elastic data compression with improved performance and space efficiency for Flash-Based storage systems. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS '17)*, pages 1109–1118. IEEE, 2017.
- [12] W. Paul Cockshott, D. McGregor, Nikolaos Kotsis, and John Wilson. Data compression in database systems. In *Ninth International Workshop on Database and Expert Systems Applications (DEXA '98)*, pages 981–990. IEEE, 1998.
- [13] Balakrishna R. Iyer and David Wilhite. Data compression support in databases. pages 695–704, 1994.
- [14] Meikel Pöss and Dmitry Potapov. Data compression in oracle. pages 937–947. Morgan Kaufmann, 2003.
- [15] Chuqing Gao, Shreya Ballijepalli, and Jianguo Wang. Revisiting B-tree compression: An experimental study. *Proceedings of the ACM on Management of Data*, 2(3):1–25, 2024.
- [16] Danny Harnik, Ronen Kat, Dmitry Sotnikov, Avishay Traeger, and Oded Margalit. To zip or not to zip: Effective resource usage for Real-Time compression. In *11th USENIX Conference on File and Storage Technologies (FAST '13)*, pages 229–241, 2013.
- [17] Danny Harnik, Ety Khaitzin, Dmitry Sotnikov, and Shai Taharlev. A fast implementation of deflate. In *Data Compression Conference (DCC '14)*, pages 223–232. IEEE, 2014.
- [18] InnoDB table compression. [Online]. <https://dev.mysql.com/doc/refman/8.4/en/innodb-table-compression.html>.
- [19] InnoDB page compression. [Online]. <https://dev.mysql.com/doc/refman/8.4/en/innodb-page-compression.html>.

- [20] Yoshinori Matsunobu, Siying Dong, and Herman Lee. MyRocks: LSM-Tree database storage engine serving facebook's social graph. *Proc. VLDB Endow.*, 13(12):3217–3230, 2020.
- [21] Dongxu Huang, Qi Liu, Qiu Cui, Zhuhe Fang, Xiaoyu Ma, Fei Xu, Li Shen, Liu Tang, Yuxing Zhou, Menglong Huang, et al. TiDB: A Raft-based HTAP database. *Proc. VLDB Endow.*, 13(12):3072–3084, 2020.
- [22] Compression in RocksDB. [Online]. <https://github.com/facebook/rocksdb/wiki/Compression>.
- [23] Compression in LevelDB. [Online]. <https://github.com/google/leveldb/blob/main/doc/index.md>.
- [24] Zhenkun Yang, Chuanhui Yang, Fusheng Han, Mingqiang Zhuang, Bing Yang, Zhifeng Yang, Xiaojun Cheng, Yuzhong Zhao, Wenhui Shi, Huafeng Xi, et al. Oceanbase: a 707 million tpmc distributed relational database system. *Proceedings of the VLDB Endowment*, 15(12):3385–3397, 2022.
- [25] Ning Zheng, Xubin Chen, Jiangpeng Li, Qi Wu, Yang Liu, Yong Peng, Fei Sun, Hao Zhong, and Tong Zhang. Re-think data management software design upon the arrival of storage hardware with Built-in transparent compression. In *12th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage '20)*. USENIX Association, 2020.
- [26] Kecheng Huang, Zhaoyan Shen, Zili Shao, Tong Zhang, and Feng Chen. Breathing new life into an old tree: Resolving logging dilemma of B+-tree on modern computational storage drives. *Proc. VLDB Endow.*, 17(2):134–147, oct 2023.
- [27] Monica Chiosa, Fabio Maschi, Ingo Müller, Gustavo Alonso, and Norman May. Hardware acceleration of compression and encryption in SAP HANA. *Proc. VLDB Endow.*, 15(12):3277–3291, 2022.
- [28] Yifan Qiao, Xubin Chen, Ning Zheng, Jiangpeng Li, Yang Liu, and Tong Zhang. Closing the B+-tree vs. LSM-tree write amplification gap on modern storage hardware with Built-in transparent compression. In *20th USENIX Conference on File and Storage Technologies (FAST '22)*, pages 69–82. USENIX Association, 2022.
- [29] Jeremy Fowers, Joo-Young Kim, Doug Burger, and Scott Hauck. A scalable High-Bandwidth architecture for lossless compression on FPGAs. In *IEEE 23rd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM '15)*, pages 52–59, 2015.
- [30] Intel QAT. [Online]. <https://www.intel.com/content/www/us/en/architecture-and-technology/intel-quick-assist-technology-overview.html>.
- [31] Weidong Zhang, Erci Xu, Qiuping Wang, Xiaolu Zhang, Yuesheng Gu, Zhenwei Lu, Tao Ouyang, Guanqun Dai, Wenwen Peng, Zhe Xu, et al. What's the story in EBS glory: Evolutions and lessons in building cloud block store. In *22nd USENIX Conference on File and Storage Technologies (FAST '24)*, pages 277–291, 2024.
- [32] Qiang Li, Qiao Xiang, Yuxin Wang, Haohao Song, Ridi Wen, Wenhui Yao, Yuanyuan Dong, Shuqi Zhao, Shuo Huang, Zhaosheng Zhu, et al. More than capacity: Performance-oriented evolution of pangu in alibaba. In *21st USENIX Conference on File and Storage Technologies (FAST '23)*, pages 331–346, 2023.
- [33] Xiang Chen, Tao Lu, Jiapin Wang, Yu Zhong, Guangchun Xie, Xueming Cao, Yuanpeng Ma, Bing Si, Feng Ding, Ying Yang, et al. HA-CSD: Host and SSD coordinated compression for capacity and performance. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS '24)*, pages 825–838. IEEE, 2024.
- [34] Gunhee Choi, Kwanghee Lee, Myunghoon Oh, Jongmoo Choi, Jhuyeong Jhin, and Yongseok Oh. A new LSM-style garbage collection scheme for ZNS SSDs. In *12th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage '20)*. USENIX Association, jul 2020.
- [35] Chen Luo and Michael J. Carey. LSM-based storage techniques: A survey. *The VLDB Journal*, 29(1):393–418, 2020.
- [36] Youyou Lu, Jiwu Shu, and Weimin Zheng. Extending the lifetime of Flash-Based storage through reducing write amplification from file systems. In *11th USENIX Conference on File and Storage Technologies (FAST '13)*, pages 257–270, 2013.
- [37] Zhe Yang, Youyou Lu, Xiaojian Liao, Youmin Chen, Junru Li, Siyu He, and Jiwu Shu. λ -IO: A unified IO stack for computational storage. In *21st USENIX Conference on File and Storage Technologies (FAST '23)*, pages 347–362. USENIX Association, 2023.
- [38] Jian Zhang, Yujie Ren, Marie Nguyen, Changwoo Min, and Sudarsun Kannan. OmniCache: Collaborative caching for Near-storage accelerators. In *22nd USENIX Conference on File and Storage Technologies (FAST '24)*, pages 35–50. USENIX Association, feb 2024.

- [39] Dongup Kwon, Dongryeong Kim, Junehyuk Boo, Won-sik Lee, and Jangwoo Kim. A fast and flexible Hardware-based virtualization mechanism for computational storage devices. In *USENIX Annual Technical Conference (USENIX ATC '21)*, pages 729–743. USENIX Association, 2021.
- [40] Wei Cao, Yang Liu, Zhushi Cheng, Ning Zheng, Wei Li, Wenjie Wu, Linqiang Ouyang, Peng Wang, Yijing Wang, Ray Kuan, Zhenjun Liu, Feng Zhu, and Tong Zhang. POLARDB meets computational storage: Efficiently support analytical workloads in Cloud-Native relational database. In *18th USENIX Conference on File and Storage Technologies (FAST '20)*, pages 29–41. USENIX Association, 2020.
- [41] Youyou Lu, Jiacheng Zhang, Zhe Yang, Liyang Pan, and Jiwu Shu. OCStore: Accelerating distributed object storage with Open-Channel SSDs. In *39th IEEE International Conference on Distributed Computing Systems (ICDCS '19)*, pages 271–281. IEEE, 2019.
- [42] Jens Axboe. Flexible I/O Tester. [Online], 2022.
- [43] Sysbench. [Online]. <https://github.com/akopytov/sysbench>.
- [44] Dominik Kempa and Nicola Prezza. At the roots of dictionary compression: String attractors. In *50th Annual ACM SIGACT Symposium on Theory of Computing (STOC '18)*, pages 827–840, 2018.
- [45] N. Jesper Larsson and Alistair Moffat. Off-line Dictionary-Based compression. *Proceedings of the IEEE*, 88(11):1722–1732, 2000.
- [46] Alexander Slesarev, Evgeniy Klyuchikov, Kirill Smirnov, and George Chernishev. Revisiting data compression in Column-Stores. In *10th International Conference on Model and Data Engineering (MEDI '21)*, pages 279–292. Springer, 2021.
- [47] Daniel J. Abadi, Samuel Madden, and Miguel Ferreira. Integrating compression and execution in Column-Oriented database systems. In *ACM SIGMOD International Conference on Management of Data (SIGMOD '06)*, pages 671–682. ACM, 2006.
- [48] Bulent Abali, Bart Blaner, John Reilly, Matthias Klein, Ashutosh Mishra, Craig B. Agricola, Bedri Sendir, Alper Buyuktosunoglu, Christian Jacobi, William J. Starke, et al. Data compression accelerator on IBM POWER9 and z15 processors: Industrial product. In *47th Annual International Symposium on Computer Architecture (ISCA '20)*, pages 1–14. IEEE, 2020.
- [49] Shucheng Wang, Ziyi Lu, Qiang Cao, Hong Jiang, Jie Yao, Yuanyuan Dong, and Puyuan Yang. BCW: Buffer-Controlled writes to HDDs for SSD-HDD hybrid storage server. In *18th USENIX Conference on File and Storage Technologies (FAST '20)*, pages 253–266, 2020.
- [50] Shengan Zheng, Morteza Hoseinzadeh, and Steven Swanson. Ziggurat: A tiered file system for Non-Volatile main memories and disks. In *17th USENIX Conference on File and Storage Technologies (FAST '19)*, pages 207–219, 2019.
- [51] Jorge Guerra, Himabindu Pucha, Joseph Glider, Wendy Belluomini, and Raju Rangaswami. Cost effective storage using extent based dynamic tiering. In *9th USENIX Conference on File and Storage Technologies (FAST '11)*, 2011.
- [52] Gong Zhang, Lawrence Chiu, and Ling Liu. Adaptive data migration in Multi-Tiered storage based cloud environment. In *3rd IEEE International Conference on Cloud Computing (CLOUD '10)*, pages 148–155. IEEE, 2010.
- [53] Ojus Thomas Lee, S. D. Madhu Kumar, and Priya Chandran. Erasure coded storage systems for cloud storage—challenges and opportunities. In *International Conference on Data Science and Engineering (ICDSE '16)*, pages 1–7. IEEE, 2016.
- [54] Jun Li and Baochun Li. Erasure coding for cloud storage systems: A survey. *Tsinghua Science and Technology*, 18(3):259–272, 2013.
- [55] Xiaolu Li, Runhui Li, Patrick P. C. Lee, and Yuchong Hu. OpenEC: Toward unified and configurable erasure coding management in distributed storage systems. In *17th USENIX Conference on File and Storage Technologies (FAST '19)*, pages 331–344, 2019.
- [56] James S. Plank. T1: Erasure codes for storage applications. In *4th USENIX Conference on File and Storage Technologies (FAST '05)*, pages 1–74, 2005.
- [57] Danny Harnik, Ety Khaitzin, and Dmitry Sotnikov. Estimating unseen Deduplication—from theory to practice. In *14th USENIX Conference on File and Storage Technologies (FAST '16)*, pages 277–290, 2016.
- [58] Kiran Srinivasan, Timothy Bisson, Garth R. Goodson, and Kaladhar Voruganti. iDedup: Latency-Aware, inline data deduplication for primary storage. In *10th USENIX Conference on File and Storage Technologies (FAST '12)*, pages 1–14, 2012.
- [59] Wen Xia, Yukun Zhou, Hong Jiang, Dan Feng, Yu Hua, Yuchong Hu, Qing Liu, and Yucheng Zhang. FastCDC: A fast and efficient Content-Defined chunking approach

for data deduplication. In *USENIX Annual Technical Conference (USENIX ATC '16)*, pages 101–114, 2016.

- [60] Zhuan Chen and Kai Shen. OrderMergeDedup: Efficient, Failure-Consistent deduplication on flash. In *14th USENIX Conference on File and Storage Technologies (FAST '16)*, pages 291–299, 2016.
- [61] Min Fu, Dan Feng, Yu Hua, Xubin He, Zuoning Chen, Wen Xia, Yucheng Zhang, and Yujuan Tan. Design tradeoffs for data deduplication performance in backup workloads. In *13th USENIX Conference on File and Storage Technologies (FAST '15)*, pages 331–344, 2015.
- [62] Qirui Yang, Runyu Jin, and Ming Zhao. SmartDedup: Optimizing deduplication for Resource-Constrained devices. In *USENIX Annual Technical Conference (USENIX ATC '19)*, pages 633–646, 2019.
- [63] Jiawei Huang, Junru Li, Qing Wang, Lijie Wen, Youyou Lu, and Erci Xu. Cablecache: In-network request deduplication for key-value stores. In *Proceedings of the 17th ACM Workshop on Hot Topics in Storage and File Systems (HotStorage'25)*, HotStorage '25, page 86–92, New York, NY, USA, 2025. Association for Computing Machinery.
- [64] Zhichao Cao, Shiyong Liu, Fenggang Wu, Guohua Wang, Bingzhe Li, and David H. C. Du. Sliding Look-Back window assisted data chunk rewriting for improving deduplication restore performance. In *17th USENIX Conference on File and Storage Technologies (FAST '19)*, pages 129–142, 2019.
- [65] Zhichao Cao, Hao Wen, Fenggang Wu, and David H. C. Du. SMRTS: A performance and Cost-Effectiveness optimized SSD-SMR tiered file system with data deduplication. In *2023 IEEE 41st International Conference on Computer Design (ICCD '23)*, pages 275–282. IEEE, 2023.